CSim Design Notes

Architecture, decisions, and performance notes

Project author + collaborative notes (Grok Build)

Started 2026-07-28 (living document; audited 2026-07-28)

Abstract

This document is the **durable design memory** for CSim (Cell Simulator): what the code is doing today, why packages and abstractions exist, and how the **render command / token** path, cellular automata rules, and headless tests fit together.

The token migration (Phases 1–6) and the performance ladder (dirty visual path, UI batching, double-buffer sim, R8 palette + PBO uploads) are reflected in the "current state" sections and the decision log. Sections marked as *inferred* were reverse-engineered; correct them freely. Append new decisions rather than silently rewriting history.

Contents

1 How to use this document

1.1 Purpose

- Survive context resets (new chats, compacted histories).
- Record **why**, not only **what**.
- Describe the **current** architecture (token renderer, CA sim, tests) and any remaining polish work.
- Separate *observed reality* from *provisional* notes.

1.2 Conventions

Status boxes
: Snapshot of maturity (prototype, shipped, production-ish).

Decision boxes
: Closed choices. Prefer adding a new decision over rewriting history silently; note supersession dates.

Open boxes
: Questions only the author can settle.

Inference boxes
: Reverse-engineered claims. Treat as provisional.

`\todoauthor{...}`
: Explicit author TODO inline in the prose.

1.3 Workflow for future sessions

1. Skim section 2, section 5, section 6, section 7, and the decision log (section 9).
2. Implement or discuss against those sections, not against chat memory alone.
3. When a decision is made, append a dated entry to the decision log.
4. When code lands, update the "current state" sections and the file map.

1.4 Building the PDF

From the repo root (requires a TeX distribution with `pdflatex` or `latexmk`):

```
cd docs
pdflatex csim-design-notes.tex
pdflatex csim-design-notes.tex   # second pass for refs/TOC
# or: latexmk -pdf csim-design-notes.tex
```

Output: `docs/csim-design-notes.pdf`.

Status / provenance

As of 2026-07-28 (post-audit): packages under `App`, `Engine`, `Game`, ...; token renderer + mock tests shipped; Canvas uses R8 cell texture + palette; double-buffered rules; headless `CSimTests`. Windows MSVC Release/Debug builds are the primary target.

2 Project overview

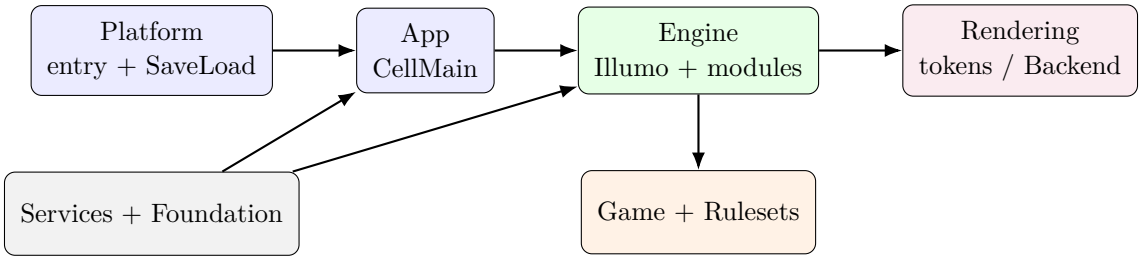
2.1 What CSim is

CSim is an interactive **cellular automata** ("cell") simulator with a 2D canvas, multiple rule sets (Game of Life, Brian's Brain, Wireworld, ...), camera navigation, edit/run modes, save/load hooks, and a small in-app command / debug surface. Rendering is currently OpenGL via GLFW/GLEW (with platform entry stubs for Linux/macOS).

Inferred from code (correct if wrong)

Historically this grew from a simpler GoL program into a mini engine shell ("Illumo"): modules, env vars, input contexts, backend interfaces, Tracy hooks. The product goal remains a cell simulator; the engine shape is partly for skill practice and multi-API future-proofing.

2.2 High-level system diagram



2.3 Runtime one-liner

Platform `main` → `CellMain` → construct `Illumo` → `Init` (services + modules) → loop { `Update`, `Render` } until window close.

2.4 Non-goals (current)

Open / needs author input

Confirm or rewrite. Draft non-goals:

- Not a general-purpose AAA renderer.
- Not multiplayer / networking.
- Not a full ECS game engine (entity scaffolding exists but is thin).
- Perfect Linux/macOS parity is secondary to Windows correctness for now.

3 Source layout

Source lives under `CSim/Source/`. Package names describe **responsibility**, not prestige.

Package	Role
`App/`	Process-level app loop after OS bootstrap (`CellMain`).
`Engine/`	Host: `Illumo`, `IModule`, context bag, debug module, entity scaffolding.
`Game/`	Domain: canvas, cell game module, editor tools (brush/cursor).
`Rulesets/`	Polymorphic CA rules; `AllSets.h` umbrella for active sets.
`Rendering/`	Window, camera, drawables, scene, RHI-ish interfaces, OpenGL backend, Mock.
`Services/`	Log, env vars, input, CLI (token UI), allocators, `SaveLoad` API.
`Foundation/`	Macros, build/sys info, math type aliases, small containers.
`Assets/`	Asset loaders (e.g. TTF).
`Platform/`	OS entry points + native dialogs / save-load.
`Tests/`	Headless suite (`CSimTests`): mock backend, rules, canvas, UI tokens.

3.1 Related trees (not packages)

- `CSim/Shader/` — GLSL assets copied to the build tree (`canvas_*.glsl`, `triangle_*.glsl`, UI shaders).
- `CSim/Assets/` — runtime files (fonts).
- `CSim/thirdparty/` — vendored deps (GLFW, GLEW, GLM, FreeType, Tracy, ...).
- `archive/` — historical code (old State pattern, dead render stacks). Not built.

3.2 Naming landmine (Windows)

Decision: MathTypes.h (2026-07-28)

Do **not** put a header named `Math.h` on a case-insensitive include path. MSVC resolves `#include <math.h>` to it and breaks the CRT/GLM. Project math aliases live in `Foundation/MathTypes.h`.

3.3 Include style (current practice)

Status / provenance

CMake adds each package directory as an include root, so both `"Logger.h"` and `"Services/Logger.h"` can work. Prefer package-prefixed includes for new code so dependencies stay visible.

4 Runtime loop and modules

4.1 CellMain

`App/CellMain.cpp` owns logger init/shutdown and the frame loop:

```
// conceptual
Logger::initLogger();
```

```
Illumo* illumo = new Illumo(argc, argv);
illumo->Init();
while (!illumo->ShouldClose()) {
  double dt = /* glfw time delta */;
  illumo->Update(dt);
  illumo->Render();
}
illumo->Shutdown();
```

4.2 Illumo as composition root

`Illumo` owns the long-lived services (`EnvVars`, `RenderWindow`, `Camera`, `Renderer`, `AssetManager`, `CommandRegistry`, `CommandLine`, `InputManager`, `Scene`) and a vector of `IModule` instances.

`IllumoContext` is a **raw-pointer bag** into those services, passed into modules at `Start`. Ownership stays on `Illumo`.

Inferred from code (correct if wrong)

Pattern: composition root + service locator-ish context struct (not a full service locator library). Modules are closer to subsystems than ECS systems.

4.3 IModule contract

```
class IModule {
public:
  virtual void Start(IllumoContext* context) = 0;
  virtual void Update(double dt) = 0;
  virtual void DispatchDrawables(Scene* scene) = 0;
  virtual void Exit() = 0;
};
```

- **Start** — bind inputs, allocate game state.
- **Update** — simulation / input.
- **DispatchDrawables** — contribute what should draw this frame.
- **Exit** — teardown module-owned state.

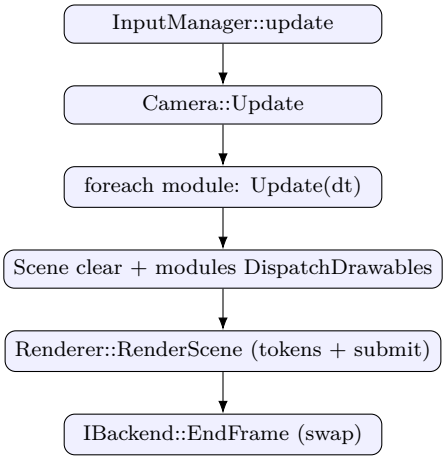
Registered modules today (from `Illumo::Init`):

- `CellGameModule` (always)
- `DebugModule` (non-`NDEBUG`)

D-E1 module registration

`Engine/Illumo.h` knows only `IModule`. `App/CellMain.cpp` constructs `Illumo`, calls `Init`, registers `CellGameModule` (and `DebugModule` in Debug), then `StartModules()`. Engine does not include Game headers.

4.4 Frame flow (conceptual)



5 Rendering — current state

Status / provenance

Token migration Phases 1–6 are complete for planned scope. Live frame path is enroll + tokens + submit via `Renderer`/`IBackend`. Drawables emit tokens through `AppendCommands`; GL draw submission is confined to `Rendering/OpenGL/` (plus window bootstrap). Headless `Rendering/Mock/MockBackend.h` powers `CSimTests`. Canvas presentation is **R8 cell texture + RGB palette** with dirty-rect PBO uploads. No second real GPU API.

5.1 What actually draws frames today

1. Modules call `DispatchDrawables` and add `DrawableBase*` pointers to `Scene` (order preserved: canvas, console, FPS, splash).
2. `Illumo::Render` drives `Renderer`: `BeginFrame` → `RenderScene` → `EndFrame` (swap).
3. `Renderer::RenderScene`: clear + viewport tokens; each drawable `AppendCommands`; if a drawable returns false, legacy immediate `Draw()` still runs (hybrid fallback; production drawables return true).
4. `IBackend::SubmitCommandQueue` → `GLDevice::ExecuteCommandQueue` resolves handles and issues GL.

5.2 Drawable design (current)

- `DrawableBase` — type-erased list entry; virtual `Draw` + virtual `AppendCommands` (default returns false). **No GL handle ownership** on the base (D-R6).
- `Drawable<Derived>` — CRTP: `Draw` → `DrawImpl` for the rare immediate path.
- Production emitters (Canvas, CommandLine, GLString/SplashText) use `AppendCommands` only; `DrawImpl` is empty/no-op.

5.3 Canvas presentation (R8 + palette)

- Domain buffer: `lifeCanvas` — one `unsigned char` per cell (state encoding: 0 = alive, 1 = dead for binary CAs; multi-state rules use additional values, e.g. Brian's Brain dying = 2).
- GPU cell texture: `GL_R8`, updated via dirty-rect `UpdateTexture` tokens (optional `srcRowStride`).
- GPU palette: 256×1 RGB texture filled by `Canvas::rebuildPalette(RuleSet*)` using `evalCell` for each state byte. Ruleset switches rebuild palette only (no full cell re-upload).
- Shaders: `Shader/canvas_vertex.glsl`, `Shader/canvas_frag.glsl` — sample R, look up palette.
- CPU float RGB fade buffers (`displayRgb`/`targetRgb`) and full-frame RGB `texCanvasBuffer` are **removed**. `cellFadeSpeed` remains as a no-op for env/console compatibility; colors snap via the palette.

5.4 Backend surface

`IBackend`: lifecycle, command queue push/submit/clear, enroll `CreateMesh`/`CreateShaderProgram`/`CreateTex` (handles = `unsigned long` table IDs).

`RenderCommand`: tagged union (`CommandType` + POD payload) including pipeline, bind, uniforms, `UpdateTexture`, `UpdateBuffer`, clear, draw (D-R1).

`GLTexture::UpdateSubImage`: PBO ping-pong + dirty-rect packing; fallback to client `glTexSubImage2D` if map fails.

`GLDevice`: bind-state tracker (skip redundant program/VAO/texture/ viewport within a submit); uniform location cache; blend-func-on-enable (D-R5).

5.5 UI tokens

- `CommandLine`: one batched `UpdateBuffer` + `DrawIndexed` when open; blend for translucent panel.
- `GLString`: geometry cache — skip VBO re-upload if content/style unchanged (FPS ∼1 Hz rebuilds).
- `SplashText`: mode label built on GLString; woken on EDIT/NORMAL.

5.6 Tests (headless)

Target `CSimTests` (alias `CSimRenderTests`); CTest names `CSimTests` / `MockBackend`. Suites: mock unit, Renderer E2E, rulesets, CellContext, canvas domain, UI tokens. No GPU context required.

5.7 Remaining pain points (non-blocking)

1. Hybrid immediate `Draw()` still exists for unmigrated stubs; production path is tokens.
2. `Renderer.h` still includes GL backend headers for production construction (test injection uses `IBackend*`).

3. `RenderWindow` still uses GLEW for context create / viewport (window bootstrap, not frame submission).

4. Platform Linux/macOS render windows may still be older than the Windows path.

5. Some scaffolding (`EntityTable`, incomplete `GLBuffer`) remains experimental.

6 Rendering — architecture and remaining work

The token architecture below is the **shipped** design. Migration phases are kept for history; treat them as complete unless a status note says otherwise.

6.1 Goals (achieved)

1. **API isolation:** frame draw submission is via tokens; OpenGL execute path lives under `Rendering/OpenGL/`.

2. **Stable front-end:** modules contribute drawables; drawables emit handles/uniforms/updates, not raw GL.

3. **Record then execute:** per-frame command stream, single submit.

4. **Resource handles:** opaque `unsigned long` table IDs; backend registries map to GL objects (D-R3).

5. **Testability:** `MockBackend` + injected `IBackend` (D-R7, D-R8).

6.2 Vocabulary

Token / RenderCommand
A discrete instruction: bind X, set uniform Y, draw Z, update texture, ...

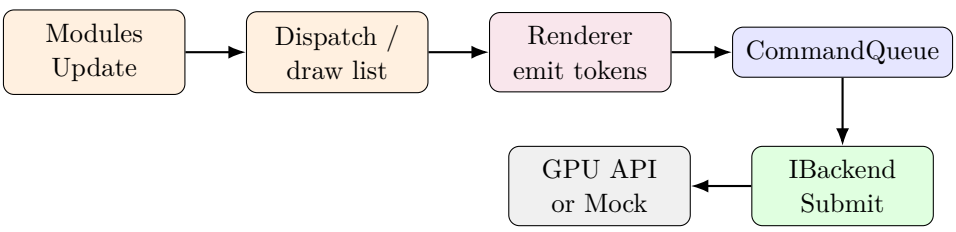
Command queue
Ordered buffer of tokens for the frame.

Backend
API-specific interpreter of tokens + owner of true GPU objects.

Enrollment
One-time (or rare) resource creation; returns a handle.

Front-end renderer
`Renderer`: enroll helpers, push tokens, `RenderScene`.

6.3 Frame pipeline (current)



6.4 Layering

```
Game / Rulesets / UI
    |  (no gl* for draw submission)
    v
Drawable::AppendCommands(Renderer*)
    |  enroll handles + push tokens
    v
CommandQueue<RenderCommand>   // tagged union payloads
    |
    v
IBackend::SubmitCommandQueue
    |
    +-- GLBackend + GLDevice   (handle resolve, PBO upload, bind tracker)
    +-- MockBackend            (tests)
```

6.5 Drawable meaning

"Drawable" means *something that can append tokens* (and optionally fall back to immediate `Draw`). GL objects live in backend registries, not on `DrawableBase`.

Canvas end state (shipped):

- Domain: `lifeCanvas`, dims, get/set, dirty rects.
- View: mesh + R8 cell texture + palette texture + canvas shaders; dirty-rect `UpdateTexture`; draw with MVP.

6.6 Migration phases (historical checklist)

1. **Phase 0 — Document & freeze vocabulary** **done**
2. **Phase 1 — Token proof path** **done** (`RenderProofQuad`, tagged union, mesh upload, handle resolve)
3. **Phase 2 — Scene through Renderer** **done**
4. **Phase 3 — Canvas tokens** **done** (later evolved to R8 + palette; see D-P4)
5. **Phase 4 — UI drawables** **done** (GLString, CommandLine, SplashText)
6. **Phase 5 — Dead path cleanup** **done** (archive under `archive/dead-render/`; pure DrawableBase)
7. **Phase 6 — Mock backend** **done** (no second real API yet)

6.7 Acceptance criteria (migration "done")

- ✓ Frame path is enroll + token queue + submit.
- ✓ Cell sim runs: edit, step/rules, camera, console, FPS, splash.
- ✓ `Renderer::RenderScene` is non-empty and used.

✓ Mock/headless tests cover tokens and domain.

- Partial: `gl*` still appears in window bootstrap and `Rendering/OpenGL/`; some non-Windows platforms lag.

6.8 Optional future work

- Generational handles / explicit destroy beyond shutdown-only (D-R4).
- Second real graphics API (only if needed).
- GPU color fade without bringing back full-grid float RGB.
- Further slim `Renderer` so production does not include GL headers in the public header (pimpl / factory).
- Linux/macOS parity with the Windows token path.

7 Game domain and rulesets

7.1 CellGameModule

Primary gameplay module. Owns a `CellContext` and an enum `CellState`:

`NORMAL` | `EDIT` | `SAVE` | `LOAD` | `EXIT`

Status / provenance

Mode handling is an **enum + switch** inside the module, not the older class-per-state hierarchy (archived under `archive/old-states/`). `E` toggles EDIT/NORMAL and wakes `SplashText`. SAVE/LOAD use Ctrl+S / Ctrl+O helpers; enum cases may still be thin. Simulation rate: env `tps` × `speedFactor`, with accumulator + step cap.

7.2 CellContext

Constructs `Canvas` from env vars (`CanvasX`, `CanvasY`) and selects a `RuleSet` from a mode string (`GAME_OF_LIFE`, `BRIANS_BRAIN`, `SEEDS`, ...). Mode changes from console/env call `setRuleSet` and `Canvas::rebuildPalette`.

7.3 Canvas

- CPU grid: `lifeCanvas` only (one byte per cell).
- Dirty flags/rects: life region, R8 upload region; skip work when idle.
- Token view: enrolled mesh, R8 cell texture, palette texture, canvas shaders (section 5.3).
- No dual float RGB buffers; presentation colors come from the palette.

7.4 Rulesets

Active rules (in `AllSets.h` / factory): Game of Life, Seeds, Brian's Brain, Highlife, Day & Night, Life Without Death. Stub headers exist for Rule 90 / 184 / Wireworld (identity `nextState`; not in AllSets).

```
class RuleSet {
  // Pure transition: cell + Moore count of alive (value==0) neighbors.
  virtual unsigned char nextState(unsigned char cell,
                                  unsigned char aliveNeighbors) const;
  virtual void evalCell(const unsigned char& target,
                        unsigned char dest[3]) const; // palette colors
  void calcGeneration(...); // double-buffer: nextGen then memcpy
};
```

Generation (perf):

1. Count-alive neighbors from raw `lifeCanvas` (toroidal, fast wrap).
2. Write `nextState` into a scratch `nextGen` buffer.
3. If any cell changed: `memcpy` back, mark sparse dirty AABB for visual/GPU upload. Still life: no dirty.

Keep rules pure

Rulesets stay free of rendering and input. They read/write cell buffers only and expose `evalCell` for palette colors. That keeps them testable (headless `TestRuleSets`).

7.5 Cell encoding

Value	Meaning (binary CAs)
0	Alive (neighbor count treats as live)
1	Dead
≥ 2	Multi-state (e.g. Brian's Brain dying = 2)

Historical neighbor tests used `!getCanvasPixel` (alive when zero).

8 Services, foundation, platform

8.1 Services

Cross-cutting facilities used by Engine/Game/Rendering:

- **Logging** — `Logger` (file + terminal; level from env).
- **Config** — `EnvVars` / `IEnvVars` (defaults filled in `Illumo::Init`: canvas size, fps/tps, mode string, ...).
- **Input** — `InputManager`, `InputContext`, `KeyCode`; action binding used by the game module. No Game types (D-E2).

- **CLI** — `CommandLine` (token UI when open; batched geometry), `CommandRegistry`, `SysCmdLine`.
- **Allocators** — pool/arena/debug allocators (experimental / learning surface; not all wired into the hot path).
- **SaveLoad API** — pure declarations; OS dialogs implemented per platform.

8.2 Foundation

Headers that should stay cheap to include: macros (`MacroDefs.h`), build info, sysinfo, `MathTypes.h`, `ArrayQueue.h`.

Open / needs author input

`MacroDefs.h` pulls Windows headers on `_WIN32`. That is convenient for `Sleep` etc., but increases include toxicity. Consider splitting platform sleep helpers out of the ubiquitous macro header.

8.3 Platform

Thin ports:

- Windows: `WinMain.cpp`, `WinSaveLoad.cpp`
- Linux: `_main.cpp`, `LinuxSaveLoad.cpp` (GTK pieces may be legacy relative to GLFW-centered shared code)
- macOS: `Main.cpp`, `MacSaveLoad.mm`

Contract: bootstrap → call `CellMain` → implement `SaveLoad::GetLoadLocation` / `GetSaveLocation`.

8.4 Third-party

Vendored under `CSim/thirdparty/`. Policy from `CONTRIBUTING.md`: new third-party deps require review/PR. Existing: GLFW, GLEW, GLM, FreeType, JSON, STB, Tracy, image libs, ...

9 Design decision log

Append-only log. Newest entries at the bottom. If a decision is reversed, add a new entry that supersedes the old id; do not silently delete history.

D-001 — Package rename (structure)

D-001 2026-07-28

Decision: Rename/split ambiguous folders to match responsibility: `Core`→`Game`, `System`→`{App,Engine,Services}`, `Init+Util`→`Foundation`, `AssetLoaders`→`Assets`.
Why: Names were lying; slowed reasoning and migrations.
Consequences: Include/CMake paths updated; archive for old States.

D-002 — MathTypes header name

D-002 2026-07-28

Decision: Project math aliases live in `MathTypes.h`, not `Math.h`.
Why: Windows case-insensitive include collision with CRT `math.h`.
Consequences: All includes updated; Foundation README notes the trap.

D-003 — Render abstraction direction

D-003 2026-07-28 (implemented; see D-R*)

Decision: Command/token submission via `IBackend` + `RenderCommand` queue; game code must not permanently depend on raw GL for draw submission.
Why: Multi-API optionality; clearer frame boundary; testability.
Consequences: Phases 1–6 complete for planned scope (section 6.6); hybrid immediate `Draw` remains only as fallback.

D-004 — Module frame hooks

D-004 inferred from IModule

Decision: Modules expose `Start`/`Update`/ `DispatchDrawables`/`Exit`.
Why: Separate simulation tick from draw contribution.
Consequences: Draw list is rebuilt or contributed each frame through dispatch (exact ownership policy still open).

D-005 — Rules as polymorphic sets

D-005 inferred

Decision: Each CA variant is a `RuleSet` subclass.
Why: Open for extension; shared generation sweep.
Consequences: Factory logic currently in `CellContext` string matching (could become a registry later).

D-006 — Mode handling (enum vs State classes)

D-006 current code; author may supersede

Decision (current): `CellState` enum inside `CellGameModule`.
Supersedes: class hierarchy under `archive/old-states/`.
Why (inferred): fewer types for a small mode set.
Open: SAVE/LOAD completeness; whether splash/global state is acceptable.

D-007 — Enrollment vs per-frame commands

D-007 visible in Renderer comments

Decision: Resource creation goes through enroll/Create* APIs; do not stuff mesh/shader creation into the per-frame token stream.
Why: Keep the frame queue about *binding and drawing*.
Consequences: Need an explicit destroy/reload story for hot-reload later.

D-008 — CONTRIBUTING constraints

D-008 from CONTRIBUTING.md

Decision: Avoid `auto`; avoid namespaces (prefer static classes/structs); no recursion; third-party via PR.
Why: Author house style / learning constraints.
Consequences: Documentation and new code should respect these unless explicitly waived in a later decision.

D-R1 — RenderCommand payload shape

D-R1 2026-07-28

Decision: `RenderCommand` uses a **simple tagged union** (`CommandType` + POD payload union), not a kitchen-sink flat struct and not a byte-stream compiler. Prefer C-style union over `std::variant`.
Why: Type-safe uniforms/clear/update data while remaining copyable for `ArrayQueue`; matches house style.
Consequences: `Rendering/RenderCommand.h` redesigned; execute path switches on type and reads the matching payload.

D-R2 — Who emits tokens

D-R2 2026-07-28

Decision: Each drawable emits its own tokens via `AppendCommands(Renderer*)` (or equivalent). `Renderer` owns frame setup (clear/viewport) and submit.
Why: Smallest strangler step from today's `DispatchDrawables` list.
Consequences: Hybrid phase may still call immediate `Draw` for unmigrated drawables after token submit.

D-R3 — Handle typing for v1

D-R3 2026-07-28

Decision: Keep `unsigned long` opaque handles with backend registries (`tableID` → GL object). Strong typedefs deferred.
Why: Minimal churn; critical fix is resolve-on-submit, not type names.
Consequences: Execute must never treat handles as raw GLuints without registry lookup.

D-R4 — Migration scope

D-R4 2026-07-28

Decision: Full migration Phases 1–5 (token proof → Scene routing → Canvas → UI drawables → cleanup). Phase 6 (mock backend) included; second real GPU API still optional.
Why: Playable strangler path to API isolation without big-bang rewrite.
Also freeze for v1: destroy resources at shutdown only; Windows GL is primary platform.
Superseded in part by D-P1/D-P4: full-frame Canvas rewrite was v1 only; dirty-rect R8 uploads are now the path.

D-R5 — Blend state on first enable

D-R5 2026-07-28

Decision: When enabling blending in `GLDevice::ApplyPipelineState`, always call `glBlendFunc` (do not rely on CPU-side factor cache matching defaults).
Why: OpenGL default is `ONE`/`ZERO`; cache defaults were SrcAlpha/OneMinusSrcAlpha so factors looked "unchanged" and console panel lost transparency after the token migration.
Consequences: Semi-transparent UI (console panel alpha) works again.

D-R6 — Dead stack cleanup (Phase 5)

D-R6 2026-07-28

Decision: Archive window `RenderQueue`, legacy `Drawable.cpp` GL helpers, empty `Transform`/`RenderContext`, and unused `CellCommandLine` under `archive/dead-render/`. `DrawableBase` is a pure list/token interface (no GL handles).
Why: Token path won; dual submission models confuse maintenance.
Consequences: `IRenderWindow::getRenderQueue` removed; frame draw list is only `Scene` + `Renderer`.

D-R7 — Mock backend for tests (Phase 6)

D-R7 2026-07-28

Decision: Add headless `MockBackend` implementing `IBackend`: record `Create*` and snapshot the command queue on `SubmitCommandQueue`. Ship as CMake target `CSimRenderTests` / CTest `MockBackend`. Do **not** add Vulkan/Metal yet.
Why: First meaningful automated render test without a GPU context; proves token vocabulary and handle plumbing independently of GL.
Consequences: CI can run `ctest -R MockBackend`; second real API remains future work after more production needs appear.

D-R8 — Inject IBackend into Renderer

D-R8 2026-07-28

Decision: `Renderer` accepts an injected `IBackend*` via constructor (`takeOwnership` flag). Production still owns a `GLBackend`. Tests inject `MockBackend` with `takeOwnership=false`.
Why: End-to-end drawable → enroll/token → submit without a GPU; keeps production construction unchanged.
Consequences: `TestRendererE2E` covers `RenderScene`, Canvas tokens, hybrid immediate drawables, and `RenderProofQuad`.

D-P1 — Dirty visual path

D-P1 2026-07-28

Decision: Canvas/CellGameModule use dirty flags so idle frames skip full-grid visual work and GPU upload.
Why: Dominant cost was recoloring + full texture upload every render frame even when the CA did not step.
Consequences: `cellsDirty`, upload pending, early-out `tickVisual`; still life after sim step does not re-dirty.

D-P2 — UI batching and geometry cache

D-P2 2026-07-28

Decision: CommandLine emits one packed `UpdateBuffer`/draw when open; GLString caches GPU geometry until content/style changes.
Why: Per-line uploads and FPS string rebuilds were measurable when the console/FPS overlay was active.
Consequences: Token stream stays small; FPS may rebuild ∼1 Hz.

D-P3 — Double-buffer CA generation

D-P3 2026-07-28

Decision: `RuleSet::calcGeneration` writes pure `nextState(cell, aliveNeighbors)` into a scratch buffer, then `memcpy`s back; neighbor counts use raw grid pointers + toroidal wrap.
Why: Correct simultaneous update; faster than per-cell `getCanvasPixel`/in-place virtual writes; sparse dirty AABB for visuals.
Consequences: All active rulesets override `nextState`; stubs use identity.

D-P4 — R8 palette + dirty-rect PBO uploads

D-P4 2026-07-28

Decision: Present cells as `GL_R8` texture of state bytes plus a 256 × 1 RGB palette (`evalCell`). Upload dirty rects via PBO ping-pong in `GLTexture::UpdateSubImage`. Drop dual float RGB staging.
Why: Large grids: bandwidth and CPU of RGB float fade dominated; palette + R8 is $O(1)$ bytes per cell on CPU.
Consequences: Colors snap (no CPU fade); ruleset change rebuilds palette only; shaders `canvas_*.glsl`; bind-state tracker in `GLDevice`.

D-E1 — Module registration lives in App

D-E1 2026-07-28

Decision: `Illumo` does not include or construct `CellGameModule`. `App/CellMain.cpp` calls `Init`, `addModule`, then `StartModules`. Debug module is registered from App under `#ifndef NDEBUG`.
Why: Engine must not depend on Game headers; product composition belongs at the application boundary.
Consequences: New products/tests can register different module sets; Engine only knows `IModule`.

D-E2 — InputManager has no Game types

D-E2 2026-07-28

Decision: Remove `CellContext*` and Game includes from `InputManager`. Also drop unused `SplashText` extern from the input header.
Why: Services must not depend on Game; input is a pure device/action service. Splash and rules stay in Game/Rendering.
Consequences: InputManager header only needs KeyCode/InputContext/GLFW.

D-E3 — EntityTable is not the cell path

D-E3 2026-07-28

Decision: Archive `EntityTable.h` and `ModuleObject.h` under `archive/dead-engine/`. Remove `entityTable` from `Scene/IllumoContext`. `ObjectID` for the lightweight scene graph lives on `SceneObject.h`.
Why: Nothing allocated or used an EntityTable; the live path is drawables + tokens, not ECS.
Consequences: Reintroduce a general object table only when a real feature needs it; document that cells are not entities.

Template for new entries

```
\subsection*{D-0XX --- short title}
\begin{decisionbox}[title={D-0XX \quad YYYY-MM-DD}]
```

```
\textbf{Decision:} ...
\textbf{Why:} ...
\textbf{Consequences:} ...
\textbf{Supersedes:} D-0YY (optional)
\end{decisionbox}
```

10 Open questions

Use this as a punch list when bouncing ideas. Strike through or move to the decision log when resolved.

Open / needs author input

1. ~~**Token payload:** flat struct vs variant vs byte stream?~~ **Resolved D-R1:** simple tagged union (C-style).
2. ~~**Handle type:** keep `unsigned long` ...?~~ **Resolved D-R3:** `unsigned long` + registries for v1.
3. ~~**Who emits tokens?** each drawable vs scene compiler?~~ **Resolved D-R2:** each drawable emits via `AppendCommands`.
4. **Resource ownership:** refcounts, generational handles, or "destroy only at shutdown" for v1? *(Working answer D-R4: shutdown-only for v1.)*
5. ~~**Canvas upload:** full texture rewrite each frame vs dirty rects?~~ **Resolved D-P1/D-P4:** dirty rects + R8 cell texture + PBO; palette rebuild on ruleset change only.
6. ~~**Illumo ↔ Game dependency:** module registration?~~ **Resolved D-E1:** App (`CellMain`) registers modules after `Init`; Engine only knows `IModule`.
7. ~~**InputManager → CellContext include?**~~ **Resolved D-E2:** InputManager is Game-free.
8. ~~**RenderQueue class:** delete, rename, or repurpose?~~ **Resolved D-R6:** archived; Scene + Renderer is the path.
9. ~~**EntityTable:** first-class for cells, or experimental?~~ **Resolved D-E3:** experimental/unused; archived under `archive/dead-engine/`. Cells are drawables, not entities.
10. **Linux/macOS:** invest now or freeze until token GL path is solid on Windows? *(Working answer D-R4: freeze until Windows token path solid — Windows token path is solid; parity still open.)*
11. **Tracy:** always-on in Debug only — any CI policy?
12. ~~**Tests:** first meaningful automated test?~~ **Resolved D-R7/D-R8:** `MockBackend` + `CSimTests` (unit, E2E, rules, canvas, UI tokens).
13. **Author intent name "Illumo":** engine codename meaning / permanence?
14. **CPU color fade:** bring back without full-grid float RGB (e.g. sparse fade or dual-state GPU mix)?
15. **Stub rules:** implement Rule 90 / 184 / Wireworld for real, or remove from the tree?

11 Glossary

Backend
Implementation of `IBackend` for a graphics API (or mock).

Canvas
Cell grid domain (`lifeCanvas`) plus token-based R8/palette view.

Command / token
One entry in the render command queue describing a GPU operation.

CRTP
Curiously Recurring Template Pattern; used by `Drawable` for optional immediate `DrawImpl`.

Dirty rect
Inclusive AABB of cells/texels that changed; used for sparse visual work and subimage uploads.

Drawable
Scene list entry that emits tokens via `AppendCommands` (preferred) or immediate `Draw` (fallback).

Enrollment
Creating GPU resources and receiving handles outside the tight frame loop.

Illumo
Engine/host object: owns services, modules, frame pumping.

IllumoContext
Non-owning pointers to services for modules.

Module
`IModule` subsystem (game, debug, ...).

Palette
256×1 RGB texture mapping cell state bytes to display colors.

PBO
Pixel buffer object; async unpack path for texture uploads.

RuleSet
Cellular automaton transition rules (`nextState` + `evalCell`).

Scene
Ordered list of drawables for the frame; not an immediate GL executor.

A Build notes

A.1 Configure & build (Windows)

```
cmake -S CSim -B build -G "Visual Studio 17 2022" -A x64
cmake --build build --config Release --target CSim
cmake --build build --config Debug --target CSim
cmake --build build --config Release --target CSimTests
```

Artifacts:

- `build/Release/CSim.exe`
- `build/Debug/CSim.exe`
- `build/Release/CSimTests.exe` (alias target `CSimRenderTests`)

A.2 Tests (headless)

```
ctest -C Release -R CSimTests --output-on-failure
# or: build/Release/CSimTests.exe
```

No OpenGL context is required (`MockBackend`).

A.3 Known setup issues

- GLEW is pulled via `thirdparty/glew-2.1.0/build/cmake`; that subtree must exist (do not delete vendored CMake helpers).
- Debug may enable `/fsanitize=address`; requires a toolchain that supports MSVC ASan.
- FreeType may warn about missing optional system deps (HarfBuzz, zlib, PNG); build can still succeed with bundled defaults.
- GLSL under `CSim/Shader/` is copied to the build tree at configure time; after adding shaders, reconfigure or copy into `build/Shader/`.

A.4 House style (CONTRIBUTING)

See `CSim/CONTRIBUTING.md`: limited `auto`, avoid namespaces, no recursion, third-party via PR.

B Key file map

Quick index for chats that need to open the right files. Not exhaustive.

B.1 Archive

- `archive/old-states/` — previous State pattern experiments.
- `archive/dead-render/` — retired immediate GL / RenderQueue paths.
- `archive/dead-engine/` — unused EntityTable / ModuleObject (D-E3).
- Analyzer/build junk may also live under `archive/`; ignore for design.

Path	Why it matters
App/CellMain.*	App loop + module registration (D-E1)
Engine/Illumo.*	Services host; addModule/StartModules; frame pump
Engine/IllumoContext.h	Service pointer bag
Engine/IModule.h	Module API
Engine/DebugModule.*	Debug overlay module (Debug builds)
Game/CellGameModule.*	Modes, input, sim tick, palette rebuild
Game/CellContext.h	Canvas + ruleset factory
Game/Canvas.*	Grid, dirty rects, R8/palette tokens
Rulesets/RuleSet.*	Double-buffer generation + nextState
Rulesets/*RuleSet.*	Concrete CA rules + evalCell colors
Rendering/Scene.h	Drawable list only
Rendering/Drawable.*	Token emitter base (no GL ownership)
Rendering/RenderCommand.h	Tagged-union tokens
Rendering/CommandQueue.*	Token buffer
Rendering/IBackend.h	Backend interface
Rendering/Renderer.h	Enroll + RenderScene + push helpers
Rendering/OpenGL/GLBackend.*	OpenGL backend registries
Rendering/OpenGL/GLDevice.*	Execute queue, bind tracker
Rendering/OpenGL/GLTexture.h	R8/RGB/RGBA + PBO UpdateSubImage
Rendering/Mock/MockBackend.h	Headless IBackend for tests
Rendering/GLString.*	FPS / text tokens + geometry cache
Rendering/SplashText.*	Mode splash (GLString wrapper)
Services/CommandLine.*	Console UI tokens (batched)
Services/InputManager.*	Input contexts/actions
Services/EnvVars.*	Runtime config
Services/Logger.*	Logging
Services/SaveLoad.h	Ported dialogs API
Platform/Windows/*	Win main + file dialogs
Foundation/MathTypes.h	GLM aliases (not Math.h!)
Tests/TestMain.cpp	Headless suite entry
CSim/Shader/canvas_*.glsl	R8 + palette presentation
CSim/CMakeLists.txt	Sources + CSimTests